CHALMERS
UNIVERSITY OF TECHNOLOGY | UNIVERSITY OF GOTHENBURG

A Chalmers University of Technology Master's thesis template for LaTeX

A Subtitle that can be Very Much Longer if Necessary

Master's thesis in Computer science and engineering

NAME FAMILYNAME

Department of Computer Science and Engineering
CHALMERS UNIVERSITY OF TECHNOLOGY
UNIVERSITY OF GOTHENBURG
Gothenburg, Sweden 2026

Master's thesis 2026

A Chalmers University of Technology Master's thesis template for LaTeX

A Subtitle that can be Very Much Longer if Necessary

NAME FAMILYNAME

UNIVERSITY OF GOTHENBURG

CHALMERS
UNIVERSITY OF TECHNOLOGY

Department of Computer Science and Engineering
Chalmers University of Technology
University of Gothenburg
Gothenburg, Sweden 2026

A Chalmers University of Technology Master's thesis template for LaTeX
A Subtitle that can be Very Much Longer if Necessary
NAME FAMILYNAME

© NAME FAMILYNAME, 2026.

Supervisor: Name, Department
Advisor: Name, Company or Institute (if applicable)
Examiner: Name, Department

Master's Thesis 2026
Department of Computer Science and Engineering
Chalmers University of Technology and University of Gothenburg
SE-412 96 Gothenburg
Telephone +46 31 772 1000

Cover: Description of the picture on the cover page (if applicable)

Typeset in LaTeX
Gothenburg, Sweden 2026

A Chalmers University of Technology Master's thesis template for LaTeX
A Subtitle that can be Very Much Longer if Necessary
NAME FAMILYNAME
Department of Computer Science and Engineering
Chalmers University of Technology and University of Gothenburg

Abstract

Abstract text about your project in Computer Science and Engineering.

Keywords: Computer, science, computer science, engineering, project, thesis.

Acknowledgements

Here, you can say thank you to your supervisor(s), company advisors and other people that supported you during your project.

Name Familyname, Gothenburg, 2026-03-27

Contents

List of Figures **x**

List of Tables **xi**

1 Introduction **1**
1.1 Aim 1
1.2 Problem 1

2 Theory **3**
2.1 In-Vehicle Systems and Event-Triggered Logging 3
2.2 Downstream Machine Learning Tasks and Data Quantity 4
2.3 Compression Theory and Traditional Methods 6
2.4 Neural Compression 8

3 Related Work **10**

4 Methods **12**
4.1 Baseline Implementation Details 12
4.2 Experiment Setup 14
4.3 Contrastive Semantic Tokenization 17

5 Results **18**

6 Conclusion **19**
6.1 Discussion 19
6.2 Conclusion 19

Bibliography **20**

A Appendix 1 **I**

List of Figures

List of Tables

1

Introduction

1.1 Aim

The aim of this project will be to investigate the use of neural compression techniques for compressing automotive time series data. The goal is to implement and test an existing architecture presented in [1], which uses vector quantization (VQ), on multiple datasources and examine the performance of the compressed data for various downstream machine learning (ML) tasks, such as predictive maintenance and anomaly detection. In order to get a better understanding of the impact of quantized latents, we will compare the implementation to a baseline with continuous latents.

The goal will be to implement and evaluate a VQ-based neural compression framework tailored for time-series data. Evaluation will be conducted by assessing the performance of the compressed representations on downstream ML tasks, comparing performance over different datasets, and analyzing relevant metrics such as compression ratio and computational efficiency.

1.2 Problem

The problem to be solved with this thesis is that of efficiently compressing vehicle telemetry data for downstream machine learning tasks. The issue is that vehicles generate a vast amount of telemetry data from various sensors and systems, which can be very challenging to store and transmit due to constraints on edge devices and communication bandwidth [2]. While techniques for data compression do exist, they are either not optimized for the specific characteristics of vehicle telemetry data or do not take into account the requirements of downstream machine learning tasks. Traditional compression techniques such as event-triggered logging and algorithmic time series compression methods have limitations when applied to vehicle telemetry data. Event-triggered logging can miss important information that does not cross predefined thresholds, leading to incomplete data for machine learning models [3]. Algorithmic compression methods often rely on manually chosen parameters and may not effectively capture the complex patterns present in vehicle telemetry data, which can negatively impact the performance of downstream machine learning tasks.

More recent approaches such as utility-aware adaptive telemetry and task-aware

compression have shown promise in addressing some of these challenges. However, these methods often require complex policy learning or adaptive algorithms that may not be feasible for real-time applications in vehicles [4]–[7].

One less explored but promising approach to compressing data which could be suitable for vehicle telemetry data is that of neural compression in combination with VQ. A system which incorporates a lightweight neural encoder and a small VQ layer should in theory be able to produce a lossy low entropy representation of the data which can be efficiently stored and transmitted. This has been achieved in non-automotive IoT applications [1] but is still relatively novel for the domain. The goal will be evaluating the performance of this approach for various datasets and downstream ML tasks in order to determine its suitability for compressing vehicle telemetry data.

Due to time constraints and the scope of this thesis, there will be several limitations to the work presented:

2

Theory

This section introduces necessary theory and establishes important context for further discussion within this thesis. The in-vehicle system and related components will be introduced, followed by an introduction to modern challenges in regard to data processing and data usage in the automotive industry. Afterwards the most important information theory concepts will be introduced, so that finally neural compression as an idea can be outlined with its most important methods and techniques.

2.1 In-Vehicle Systems and Event-Triggered Logging

The In-Vehicle Embedded System An embedded system is a computer system integrated into a physical device or product to perform a dedicated function as part of a larger system [8, pp. vii–x, 1–3]. The in-vehicle embedded system is a specialized embedded system integrated within a vehicle. It is essential for real-time functions like controlling, monitoring, and enhancing various automotive operations [9, pp. 1-1–1-5]. These in-vehicle embedded systems, like embedded systems in general, typically consist of both hardware and software components, such as electronic control units (ECUs), sensors, actuators, and communication interfaces [8, pp. 3–4], [9, pp. 1-3–1-10].

ECUs serve as the primary processing nodes in in-vehicle embedded systems. They are responsible for executing control algorithms on sensor inputs and issuing commands to actuators. They also handle local control logic while coordinating via in-vehicle networks to form a distributed system [9, pp. 1-8–1-19]. An example of how ECUs work together to create the in-vehicle embedded system is illustrated by [9, pp. 1-8–1-9]. In the example an ECU coordinates a request, e.g., locking or unlocking a door, while the controller realize the request on the physical system, which is controlled by another ECU. The resulting system is the main generator of telemetry data within the vehicle. Vehicle telemetry data, usually defined as data that is produced by the vehicle and sent off-board, is key to improving important aspects of these vehicles like safety [10, pp. 1–5]. This vehicle telemetry data is at the core of this thesis.

In-Vehicle Networks As mentioned above, in-vehicle networks serve as the connectors between the ECUs of the in-vehicle embedded system. Among these in-vehicle networks Control Area Networks (CAN), Local Interconnect Networks (LIN), FlexRay,

and Ethernet are popular representatives that enable efficient communication and coordination among different vehicle subsystems [9, pp. 1-2–1-5].

From a technical perspective, ECUs connect to these networks via dedicated communication services. The communication services are tasked with providing a uniform interface to the network, while hiding protocol and message properties from the application [11, p. 48]. The in-vehicle network protocols are categorized into three classes, A, B, and C. LIN falls into category A with a bit rate below 10 kbps and application in sensor and actuator networks. CAN-B is a representative of category B with a bit rate between 10–500 kbps. Category B networks serve vehicle-centric domains and body electronics. Category C in-vehicle networks like FlexRay or CAN-C are defined by having a bit rate of lower than 1 Mbps and used in safety-relevant systems [9, pp. 1-12–1-13]. Despite their effectiveness in their respective use-cases, all in-vehicle networks must contend with fundamental bandwidth limitations that constrain data throughput across the vehicle [9, pp. 1-18–1-19].

Event-Triggered and Time-Triggered Logging Event-triggered logging and diagnostic frameworks, which record data only when specific error codes occur or threshold crossings are detected, are widely adopted to reduce data transmission and avoid bus saturation in complex systems such as the in-vehicle embedded system [12, pp. 7–13]. According to the AUTOSAR Diagnostic Log and Trace (DLT) standard, this mechanism can be implemented as follows: When software components or Basic Software modules report errors via the Default Error Tracer, the DLT module receives these via a specified function and stores timestamped log/trace messages to flash storage [12, p. 25]. Non-critical events are filtered during normal operation [12, pp. 30–35].

Time-triggered logging serves as an alternative to event-triggered logging and works by setting predetermined points in time where data is transmitted. The validity of this strategy is easily monitored, as the time-frame scheduling is predictable and missing messages are immediately identified [9, pp. 4-5]. Although Navet discusses these paradigms in the context of communication scheduling, similar principles apply to diagnostic event- and time-based mechanisms, which leads to similar struggles in these systems.

While these approaches have been effective in reducing the data load of the in-vehicle networks, they come with major limitations. Two immediate issues these diagnostic frameworks struggle with is presenting an unbiased and comprehensive monitoring of the system, while having to carefully tune event thresholds and diagnostic parameters [9, pp. 4-5–4-6], [13]. In the next section we will go further into how these limitations are accentuated by recent developments.

2.2 Downstream Machine Learning Tasks and Data Quantity

Two developments in recent years further underline the shortcomings of event-triggered logging in automotive systems: the massive increase in signal-based data

in the in-vehicle network and the growing relevance of downstream machine learning (ML) tasks.

Data Quantity in Modern Vehicles Recent industry and research reports indicate that the data quantity generated by sensors and the amount of interconnectivity in vehicles is growing at an extremely rapid pace. According to a 2021 automotive electronics report, by 2030, about 95% of new vehicles will be connected, up from around 50% today [14]. Another report states that even today, a single car can generate up to 1 terabyte (TB) of data per hour from its sensors [2]. This explosive growth is driven by the increasing number and sophistication of sensors—such as cameras, radars, and lidars—required for advanced safety and autonomous driving features, with the complexity and volume of data presenting significant challenges for storage, processing, and transmission within embedded automotive systems [2], [15, pp. 3–5]. While much of the focus in research and reporting on this issue centers on autonomous driving technologies like lidars and cameras, it is important to note that the core of the in-vehicle system, the ECU, is also a major contributor to the data quantity issue. This implies that not only autonomous driving technologies but modern vehicles in general are facing these challenges [15, pp. 3–4].

Downstream ML Tasks Modern vehicles increasingly rely on data-driven intelligence to enhance safety, reliability, and efficiency. As there currently exist no conclusive definition of these tasks, we will refer to them as downstream ML tasks within the context of this thesis. These downstream ML tasks we define as the offline use of collected vehicle data for the training of ML models for various tasks. This is distinctly different from real-time perception and control systems. Downstream ML tasks have become central to automotive-system design. Theissler et al. [16, p. 2] mention three reasons for the rise of importance of these downstream ML tasks: data availability, breakthroughs in algorithm development and the advancements in computational power. Prominent examples of downstream ML tasks in the automotive context include: Predictive maintenance tasks aim to predict the expected time of a failure, thereby helping to estimate remaining functionality of components or systems [16, p. 3]. Anomaly and intrusion detection tasks want to find anomalies or outliers in data points which is especially relevant in fragile systems like the CAN network [17, pp. 1–3]. Fleet-level management operations like fuel consumption that aim to better utilize important assets in vehicle systems [18, pp. 1–2]. Utility loss in the datasets used to train the models for these tasks translates directly to degraded model performance, increased operational costs, and missed business opportunities [15, pp. 4–5]. This further underlines the importance of a comprehensive data logging strategy.

There exists relatively little research on the use of event-triggered logging systems as a compression strategy and it's impact on downstream ML tasks. It is therefore difficult to definitively say that event-triggered logging systems specifically produce data with lower utility. Studies have however shown that similar utility agnostic data reduction strategies can have negative impacts on downstream utility in the domain of time-series data. One such example comes from a study by [19, pp. 3179–3185] where the authors show that a 10-minute aggregation of the data leads to degraded utility for downstream ML tasks. In this specific case, the F1-score of the

predictive downstream model was reduced by 13% when trained on the aggregated data compared to the original dataset. The same study also shows that a model based lossy compression technique can be implemented without significant utility loss which suggests that more strategic data reduction techniques can retain more utility for downstream ML tasks. Another example comes from a study by [20, pp. 746–751] which similarly found that strategic compression can be applied without loss of classifiability of the data.

2.3 Compression Theory and Traditional Methods

Information Theory Because information theory provides the basis for compression methods, its core concepts are introduced here before discussing compression itself. Entropy is key to information theory. It measures the uncertainty of a random variable and is computed from its probability distribution. More specifically, the uncertainty in a probability distribution is the expected negative log-probability of its outcomes. The entropy of a discrete distribution $p = (p_i)_i$ is defined as

$$H(p) = -\sum_i p_i \log p_i,$$

where the base of the logarithm determines the unit [21].

To illustrate with a weather example, suppose the true probabilities of "rain" and "shine" are $p_1 = 0.3$ and $p_2 = 0.7$, respectively. Then (using base-2 logs),

$$H(p) = -(p_1 \log p_1 + p_2 \log p_2) \approx 0.61.$$

Suppose the measurements are instead from Abu Dhabi, where the probabilities might be $p_1 = 0.01$ and $p_2 = 0.99$. The entropy is now approximately 0.06. There is thus much less uncertainty about any given day compared to a place where it rains 30% of the time. In this way, information entropy quantifies the inherent uncertainty in a distribution of events.

Entropy is central to compression because it indicates how compressible data is. High-entropy data is less predictable and therefore harder to compress, while low-entropy data has redundant patterns and is therefore easier to compress [21].

Compression Compression, as originated in information theory by [21], is the process of encoding information using fewer bits than the original representation. Compression techniques can be broadly categorized into lossless and lossy methods. Lossless compression is based on two principles: distribution modeling, sometimes called entropy modeling, and entropy coding. Entropy modeling involves creating a probabilistic representation of the data, while entropy coding assigns shorter codes to more frequent symbols based on their probabilities, thereby minimizing the average code length. Entropy serves as the lower bound (in bits per symbol) for any lossless compression scheme. Lossy compression allows for some loss of information in exchange for higher compression ratios. This is typically achieved through techniques such as transform coding and quantization [22]. Closely connected to lossy compression techniques is the term "compression ratio". Compression ratio is

defined as the size of uncompressed data divided by the size of compressed data [23]. For the purpose of this thesis, the focus will be on lossy compression as this is more suitable for common downstream ML tasks where some loss of fidelity is acceptable as long as the relevant information for the task is preserved.

Rate-Distortion Theory Rate-distortion theory provides the fundamental framework for lossy compression, quantifying the minimum bitrate $R(D)$ required to represent data with acceptable distortion at most D. Building on entropy and mutual information introduced earlier, it formalizes the fundamental tradeoff between rate and fidelity central to lossy compression. The rate-distortion function is formally defined as

$$R(D) = \min_{p(\hat{X}|X):\ E[d(X,\hat{X})] \leq D} I(X;\hat{X}),$$

where $d(x, \hat{x})$ is a distortion measure, e.g., squared error, $E[\cdot]$ is the average over all data points or expectation, and the minimum bitrate is computed over all distributions $p(\hat{X}|X)$ achieving average distortion at most D. Shannon's rate-distortion theorem proves this isn't just a theoretical lower bound and practical codes exist that get arbitrarily close to $R(D)$ bits/symbols while keeping distortion below D. [24].

Distortion measures quantify the cost of representation between original data X and its compressed reconstruction $\hat{X}$. They are essential in rate-distortion theory, where the goal is to minimize bitrate subject to $E[d(X, \hat{X})] \leq D$ for a chosen distortion threshold D. Widely used examples for distortion measures include the squared error (MSE) or the Hamming distance. MSE is defined as $d(x, \hat{x}) = (x - \hat{x})^2$, and is often used for numerical/time-series data due to its mathematical tractability. The Hamming distance on the other hand is defined as $d(x, \hat{x}) = \sum_i \mathbf{1}\{x_i \neq \hat{x}_i\}$, and is used for binary or categorical data [25, pp. 304–305].

In the ML context, particularly in automotive systems, or in fact Internet-of-Things (IoT) systems in general, rate-distortion analysis denotes the trade-off between efficiently handling large quantities of data and maximizing model performance. So in this sense, rate-distortion theory gives the theoretical framework to the downstream ML task challenges described in Section 2.2. Traditional lossy compression techniques can reduce data volume, but often at the cost of losing critical information necessary for accurate ML tasks such as predictive maintenance, anomaly detection, and fleet analytics [26]. The impact of this trade-off is well-documented in the literature. Cuza et al. [27] for example, study the impact of lossy compression techniques on time series forecasting tasks and observe a constant trade-off between compression ratio and forecasting accuracy. It is important to note, that, while the authors of this paper see impact of compression ratio on task performance, it is not necessarily a negative impact and is dependent on various factors [27, pp. 654–655].

Traditional Compression Methods Traditional compression methods, based on information theory principles, often fall short in automotive applications, especially as a precursor for downstream ML tasks. For video/image compression, traditional methods like JPEG are optimized for human perception (e.g., visual quality) rather than ML tasks or efficient downstream data use [28, p. 12]. For time series data, algorithmic approaches like CHIMP or Gorilla depend on manually chosen parameters

like window size and are sensitive to data characteristics such as entropy and signal variability. This limits their effectiveness in capturing the nuances required for accurate ML model performance in automotive contexts [29, pp. 37–38 and 47]. These algorithmic approaches were investigated by Johnsson [29] in a previous Master's thesis project. This work builds upon this thesis by exploring neural alternatives for vehicle telemetry compression. These neural alternatives are more flexible and can be optimized for specific downstream tasks and data characteristics, potentially offering better performance in terms of the rate-distortion trade-off for ML tasks. As previously shown by [19, pp. 3179–3185], model aware compression techniques have the potential to retain more task relevant information compared to traditional methods, which is key for achieving better performance for downstream ML tasks while still maintaining significant compression ratios.

2.4 Neural Compression

Information Bottleneck Theory While the theory presented by [21] gives a theoretical framework for understanding the limits of compression, it falls short when it comes to learned compression as it only considers the maximum compression ratio for a given distortion level, without considering the specific characteristics of the data or the downstream tasks. This shortcoming is addressed in the information bottleneck (IB) theory, which presents a more nuanced approach to the issue of compression. In order to maintain relevance for downstream tasks, we need to preserve as much relevant information as possible about the target label, not about the input data. This is the core idea of the IB theory, which formalizes the trade-off between compression and relevance as:

$$\min_{p(\hat{X}|X)} I(X;\hat{X}) - \beta I(Y;\hat{X}),$$

where $I(X;\hat{X})$ is the mutual information between the input data X and its compressed representation $\hat{X}$, $I(Y;\hat{X})$ is the mutual information between the target label Y and the compressed representation $\hat{X}$, and β is a Lagrange multiplier that controls the trade-off between compression and relevance [30].

Early work on attempting to understand the learning dynamics of deep neural networks (DNNs) have applied the IB theory in an attempt to explain how DNNs learn compressed representations of data while preserving relevant information for specific tasks [31, pp. 1-5]. The core of this idea is that the goal of any supervised learning algorithm is to capture a minimally sufficient statistic in the input required to express the output. This mirrors the goal of of IB compression which claims that there exists a maximally compressed representation of the input which retains the maximum amount of information about the target label. Other works such as [32] have attempted to show that DNNs both fit the input in order to increase the mutual information between the target and the latent representation $I(T;Y)$ while also compressing the input by lowering the mutual information between the input and the latent representation $I(X;T)$. This work has however been criticized for its methodology and computation of mutual information. Much of the issue comes down

to the fact that the layers of a DNN cannot lower mutual information, but rather just learns to map the input to consistent output values. In order for a network to be able to discard information and thereby compress the input on an IB sense, additional measures are required. Future work generally make a distinction between IB "compression" (which requires noise/stochasticity) and "dimensionality reduction" (which can occur in deterministic networks such as pure DNNs) [33]

Neural Compression While there have been various approaches to achieving true compression of information in the context of IB using neural networks, such as the variational information bottleneck (VIB) architecture [34], the approach which will be presented in this thesis extends the normal encoder with a vector quantization (VQ) bottleneck in order to minimize the entropy. This idea is conceptually more similar to methods such as those presented in [35, pp. 1611–1630]. The VQ layer acts as a deterministic information bottleneck partitioning the input space X into regions that map to the same discrete code t, discarding intra-cluster variations. By binning the continuous signal to a finite set of codes, the entropy of the signal is bounded by the number of bins. As mentioned in [33], the encoder cannot exhibit decreasing $I(X;T)$ without noise or stochasticity since the entropy is constant $H(T \parallel X)$. It can however create sharper decision boundaries which allows the VQ bottleneck to more effectively quantize the input without discarding relevant information.

Neural compression has been shown to be an effective strategy for handling the data rate-utility trade-off in various domains. Studies as early as 2019 have shown that neural compression methods can outperform traditional compression techniques for image and video data, especially at low bitrates [5]. The same has been shown for time series data [4], [7]. This approach has also been shown to be effective in resource-constrained environments, such as IoT systems, where the ability to retain task-relevant information while reducing data volume is crucial [3] and [36].

3

Related Work

NOTE: FROM THIS POINT ON, THE SECTIONS ARE NOT THESIS-READY AND ONLY FOR THE PURPOSE OF THE HALFTIME REPORT.

Within the neural compression research community, there have been a few distinct research directions that are important to consider for this thesis. Within this chapter we will discuss them, show relevant examples, and pinpoint, where the gaps lie that this thesis will fill.

Edge-Optimized Neural Compression The overwhelming majority of "edge-optimized neural compression" research focuses on compressing the ML models themselves for deployment, not compressing the input data. See this for more information [37]. There is one notable exception from this, which we will discuss later.

Time-Series Neural Compression: Research on neural compression for time-series data exists, but usually not optimized for edge deployment. See this for an example of this kind of research [38]. Again, there is one notable exception.

Audio/ Speech Neural Compression: The audio/speech community has extensive research into vector quantization methods to enable neural compression of this kind of data. Multiple papers have contributed to the space here and introduced the residual vector quantization (RVQ) architecture:

- SoundStream (Google, 2021): first major use of RVQ for audio compression [39, pp. 495–507].
- EnCodec (Meta, 2022): refined RVQ for high-quality audio [40].
- WaveTokenizer (2024): newest RVQ audio codec [41].

EdgeCodec: EdgeCodec takes this idea of using RVQ from audio/speech processing and applies it to time-series data (aerospace sensor data) while implementing the architecture in a way to be edge-optimized (lightweight encoder, heavy decoder) [1]. EdgeCodec therefore is a notable outlier in the research community and addresses a lot of the challenges we introduced in our background section. They do however have one notable limitation, they only focus on reconstruction error and not downstream task utility.

=> Key insight: Neural compression as a concept, using VQ-based quantization, and using a lightweight encoder have all been explored before. What is missing, is a

neural compression method that optimizes for downstream ML task utility rather than just reconstruction quality.

Based on this, we want to explore the impact of existing solutions on downstream ML task performance. For this, we implement baselines, observe their performance when applied to automotive telemetry data, and possibly improve upon existing solutions to optimize for downstream utility retention.

4

Methods

NOTE: OUR METHODOLOGY CHANGED A LITTLE COMPARED TO WHAT WE OUTLINED IN THE PROJECT PLANING REPORT. THE MAIN REASON FOR THIS IS THE FINDING OF THE EDGECODEC PAPER. BECAUSE THIS PAPER ALREADY ANSWERS SOME OF OUR RESEARCH QUESTION THE FOCUS WILL BE MORE ON THE ANALYSIS OF THEIR SOLUTION (AND A CONTINUOUS LATENT VARIENT) IN REGARDS OF DOWNSTREAM TASK UTILITY AND POSSIBLE IMPROVEMENTS TO OPTIMIZE SPECIFICALLY FOR THIS.

Within this chapter, we will discuss the methodology employed in our project to achieve the outlined aim. Specifically, we will discuss the architecture of the established compression models and possible improvements upon them. We additionally discuss the experiment setup we will use to analyze these approaches.

4.1 Baseline Implementation Details

First, we want to outline the architectures of the established compression methods we will use as baselines.

Baseline 1: Continuous Latent Autoencoder We will use the continuous latent autoencoder approach introduced by Zheng et. al. [4] as an inspiration for our autoencoder baseline model. It is important to note, that the paper introduces a sophisticated framework consisting of two models, the TCN-RNN model and the TCN-ARNN model, as well as a model selection mechanism. For simplicity sake, we will only focus on one of these models, the TCN-RNN model. The TCN-RNN model described in the paper is a representative of state-of-the-art continuous-latent neural time-series compression methods. Compared to vanilla autoencoders, the models architecture is explicitly designed for long temporal dependencies and has demonstrated stronger rate-distortion performance. This allows us to evaluate our tokenization-based approach against a competitive neural compressor rather than a toy model. Another point to note about the TCN-RNN model, is its costly architectural design, namely deep temporal convolutional stacks with large receptive fields, recurrent (often sequential) decoding, and dense continuous latent representations. These components typically lead to increased FLOPs, higher activation memory, and longer inference latency compared to discrete, token-based compression methods.

On a high level, the TCN-RNN architecture consists of the following components:

- Input handling and preprocessing
- TCN encoder
- RNN bottleneck
- GRU decoder

For the input handling and preprocessing it is important that the time-series data is segmented into fixed-length windows. These windows are treated as inputs to the model. Overlapping windows can be used to increase the effective training data size and improve reconstruction quality at window boundaries. Additionally, each channel is normalized independently using statistics computed on the training set.

The TCN encoder consists of a stack of Temporal Convolutional Network (TCN) blocks with increasing dilation factors. Each TCN block contains: A 1D convolution operating along the temporal dimension, normalization (BatchNorm or LayerNorm), a nonlinear activation function (ReLU or LeakyReLU), optional dropout for regularization, and a residual connection to stabilize training. The outcome is a continuous latent representation whose size is controlled by the downsampling factor and channel width.

The encoder output is processed by a recurrent neural network, specifically a gated recurrent unit (GRU), the RNN bottleneck. The RNN captures longer-range temporal dependencies that are not easily modeled by convolutions alone. At this stage, we produce a sequence of hidden states that serves as the core compressed signal. The hidden state dimensionality and number of layers define the strength of the bottleneck. The compression is achieved implicitly by restricting temporal resolution and latent dimensionality rather than by discretization. Afterwards, the RNN decoder reconstructs a latent sequence from the bottleneck representation. The decoder mirrors the bottleneck RNN in depth and hidden size.

Finally, a mirrored GRU stack upsamples the latent sequence back to the original temporal resolution. Transposed convolutions or interpolation-based upsampling are used. A final linear projection maps features back to the original channel dimension. The overall training objective is the reconstruction loss (e.g., MSE or MAE) between input and reconstructed signal.

Current Status and Preliminary Results We have successfully replicated the baseline autoencoder. We have implemented the encoder, bottleneck, and decoder according to the paper's specification and achieve comparable results. For now, we have tested this replication on the UCI Appliances Energy Prediction dataset and tried to compress and reconstruct two signals, "Appliances" and "Windspeed" (same as Zheng et. al. [4]). Zheng et. al. report a reconstruction error of 0.178 for this dataset, these signals, their implementation, and a sequence length of 150. For the same setup, but using our replication, we get a reconstruction error of 0.165. We therefore conclude, that the replication was successful, and this is a valid baseline to be used in further experiments.

Baseline 2: EdgeCodec The EdgeCodec implementation leverages residual RVQ and a heavyweight decoder to enable a lightweight encoder architecture, which is edge-optimized. The encoder used by the authors works through progressive downsampling through dilated convolutions in residual units, enabling 32x compression. The RVQ utilized for the EdgeCoded architecture is a 4-stage hierarchical quantizer. This means that each stage quantizes residuals from the previous stage. As mentioned, the decoder is intentionally heavy and uses a ELU activation function with transposed convolutional layers for upsampling [1].

Current Status and Preliminary Results We are currently in the process of replicating the EdgeCodec architecture. Fortunately the authors provide a Github repository with relevant code, so the implementation is going along smoothly.

4.2 Experiment Setup

To compare and evaluate the two methods outlined above, we will conduct a series of experiments. In this subsection we outline how we ensure a fair comparison, what metrics we will use, and what data we will use for the experiments.

Comparison and Metrics: There are three dimensions along which we want to compare the two compression methods. First, we want to evaluate the *compression performance* of both methods. This includes the metrics compression rate and reconstruction error. These metrics align with the evaluation standards in the neural compression literature. We will measure these metrics by training the two models on a reconstruction task and evaluating the reconstruction quality and effective compression rate. Second, we want to evaluate the *downstream task performance* when using compressed representations from both methods. This includes metrics such as accuracy for classification tasks or mean squared error for regression tasks. This evaluation reflects the practical utility of the compressed data for real-world applications. Third, we want to compare the *model efficiency* of both methods. This includes metrics such as parameter count, computational cost (FLOPs), inference latency, and memory footprint. These metrics are important for assessing the feasibility of deploying the compression methods in resource-constrained environments.

An important note to make regarding comparability of these two metrics, especially regarding the downstream task performance, is that the two methods produce different types of compressed representations. The TCN-RNN baseline produces a continuous latent representation while the EdgeCodec implementation produces discrete latents. To ensure a fair comparison, we will need to design a lightweight unobtrusive adapter for each representation type that maps them into a common format suitable for the downstream model. The adapters should have minimal capacity to avoid introducing bias. This way, we can isolate the effect of the compression method itself on downstream performance. The envisioned design for the adapters is as follows:

- The adapter for the continuous latent representation will take the sequence of real-valued vectors output by the TCN-RNN model, average them over time

to produce a single vector, and then pass this vector through a linear layer to match the input size of the downstream model.

- The adapter for the discrete latent representation will take the sequence of codebook IDs produced by the EdgeCodec model, map each ID to a small vector using an embedding lookup, average these embeddings over time to produce a single vector, and then pass this vector through a linear layer to match the input size of the downstream model.

Data: The goal is to use industry relevant data, while also ensuring that the methods are tested on publicly available data. For that reason, we will focus on four datasets: Two Volvo datasets, the Volvo test fleet dataset and the Volvo electric mobility (EMOB) dataset, and two publicly available dataset, one from each baseline paper. Each of these datasets contains time-series data collected from e.g. vehicles, including sensor readings, telemetry data, and operational parameters.

Downstream Models: Comparing the compression methods based on downstream task performance requires downstream models. These models need to be simple enough to let us easily evaluate the compression methods used to compress the data which is fed into these models, but also still relevant to actual use cases. For this reason we will use different downstream models across Volvo and publicly available data and different use cases to test the capabilities of the described compression methods.

Current Status and Preliminary Results We have implemented relevant dataloaders and downstream models for our use cases. We also setup repositories for the data interfaces, the downstream models, and the compression methods discussed to prepare for the experimentation phase.

We implemented four datasets within our data interface for this thesis. Each dataset includes a torch "Datasets" and a pytorchlightning "LightningDataModule". Between these two components, the torch "Datasets" are responsible for data preparation steps like connecting to the data source, loading the data, filtering the data, computing global statistics, and creating windows. The pytorchlightning "LightningDataModule" wrap around this logic and split the data to produce the machine learning-ready test/val/train datasets. The following four data interfaces were implemented:

- Trino Dataset: This dataset connects to the Volvo internal Trino databases to load test fleet vehicle telemetry data. Within this dataset we implement trip-based splitting to avoid data-leakage.

- Emob Dataset: Another Volvo internal dataset, that provides battery lifecycle data through a .csv file. The data originates from the E-Mobility (EMOB) team within Volvo. Here we implement cycle-based splitting to avoid data-leakage.

- Time Series Text Dataset: This is a more general dataset that works with any text-based time-series data with chronological ordering. For our purposes it is used to provide the datasets mentioned by [4]. We want to use these datasets to ensure a fair comparison with their method. It is to be evaluated, if this dataset is also suitable for the data used by the EdgeCodec paper.

In the second repository, we implemented the downstream models that will serve as use cases to test the compression methods or more specifically the compressed data. The following downstream models were implemented:

- Fuel Prediction: The first model to be implemented was a simple cnn-regressor for the fuel prediction use case. For this we use the testfleet data from the Trino Dataset. Specifically, we the following signals: "enginespeed", "grosscombinationvehicleweight", "actualenginepercenttorque", "transmissioncurrentgear", "roadinclination", "acceleratorpedalposition1", "breakpedalposition", "highresenginetotalfuelused". The last of these is the target signal. Unfortunately we had some trouble with the "highresenginetotalfuelused", which turned out to be quite unreliable. For this reason, the driver input prediction use case was implemented as an alternative.

- Driver Input Prediction: The driver input prediction use case uses the same cnn-regressor as the fuel prediction use case. We also use roughly the same data, just with different target signals. Here the used signals are: "enginespeed", "grosscombinationvehicleweight", "actualenginepercenttorque", "transmissioncurrentgear", "acceleratorpedalposition1", "breakpedalposition", with the last two being the target signals. This is much more reliably supported by our data, but obviously less common in the automotive context. A final decision on which of if we will use this use case or the fuel prediction one has not yet been made.

- EMOB Anomaly Detection: For the EMOB use case, we decided to implement a simple cnn-classifier to do anomaly classification. This is a fairly straight forward use case that uses current, voltage, and temperature signals to predict anomalies. The only important note to make here, is, that the data was labeled using a interquartile range (IQR) method that was not implemented by us, but by the Volvo team.

- Power Consumption: For this we use the Time Series Text Dataset to load the publicly available UCI Houshold Electric Power Consumption dataset. With this, we predict the signal "Global_active_power" based on various other signals. We want to include this use case for two reasons. One, to have a publicly available benchmark dataset for our compression methods. And two, because our baseline paper uses it. This use case is pretty interchangeable with all datasets used by Zheng et. al. [4]. A final decision on which of the datasets to use has not been made.

This means, that, once the EdgeCodec and relevant datasets are implement, we are ready to start the experimentation phase and use the implemented data interfaces to train the compression methods outlined, observe compression performance and model efficiency, save the resulting latent representations, and train the implemented downstream models to compare the loss in downstream task utility.

4.3 Contrastive Semantic Tokenization

We expect to find, that downstream utility is compromised to an extend when utilizing the EdgeCoded compressor. Here, we want to try to improve upon their solution by using contrastive semantic tokenization to create interpretable, behavior-aligned tokens. The basic idea is to use contrastive learning, which shapes the embedding space by pulling similar signal behaviors together and pushing different behaviors apart. This creates exlpicit semantic structure, which we hope to leverage to create semantic tokens.

Contrastive semantiv tokenization has seen success in image compression, validating our use of this particular technique [42]. Contrastive learning on its own has also been successfully applied to time series forecasting and classification. We therefore argue, that temporal patterns in time series data are similarly clusterable as image data, and can be similarly compressed.

5

Results

Describe you results. Use tables, diagrams etc. for illustration.

6

Conclusion

You may consider to instead divide this chapter into discussion of the results and a summary.

6.1 Discussion

6.2 Conclusion

Bibliography

[1] B. Hodo, T. Polonelli, A. Moallemi, L. Benini, and M. Magno, "Edgecodec: Onboard lightweight high fidelity neural compressor with residual vector quantization," in *2025 10th International Workshop on Advances in Sensors and Interfaces (IWASI)*, IEEE, 2025, pp. 1–6.

[2] R. Samantaray, "Adas sensor data handling in the world of autonomous mobility," *SAE Technical Paper Series*, 2023. DOI: 10.4271/2023-01-0993.

[3] J. Azar, A. Makhoul, R. Couturier, and J. Demerjian, "Robust iot time series classification with data compression and deep learning," *Neurocomputing*, vol. 398, Feb. 2020. DOI: 10.1016/j.neucom.2020.02.097.

[4] Z. Zheng and Z. Zhang, "A temporal convolutional recurrent autoencoder based framework for compressing time series data," *Applied Soft Computing*, vol. 147, p. 110 797, 2023, ISSN: 1568-4946. DOI: https://doi.org/10.1016/j.asoc.2023.110797. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1568494623008153.

[5] J. Löhdefink, A. Bär, N. M. Schmidt, F. Hüger, P. Schlicht, and T. Fingscheidt, "Gan- vs. jpeg2000 image compression for distributed automotive perception: Higher peak snr does not mean better semantic segmentation," *arXiv preprint arXiv:1902.04311*, 2019. DOI: arXiv:1902.04311v1.

[6] M. Kawawa-Beaudan, R. Roggenkemper, and A. Zakhor, "Recognition-aware learned image compression," *Electronic Imaging*, vol. 34, no. 14, pp. 220-1-220–5, Jan. 2022, ISSN: 2470-1173. DOI: 10.2352/ei.2022.34.14.coimg-220. [Online]. Available: http://dx.doi.org/10.2352/EI.2022.34.14.COIMG-220.

[7] J. Liu, P. Djukic, M. Kulhandjian, and B. Kantarci, *Deep dict: Deep learning-based lossy time series compressor for iot data*, 2024. arXiv: 2401.10396 [eess.SP]. [Online]. Available: https://arxiv.org/abs/2401.10396.

[8] E. A. Lee and S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*, 2nd. Cambridge, MA, USA: MIT Press, 2017, Available at https://ptolemy.berkeley.edu/books/leeseshia/.

[9] N. Navet and F. Simonot-Lion, Eds., *Automotive Embedded Systems Handbook*. Boca Raton, FL, USA: CRC Press, 2009, ISBN: 978-0-8493-8026-6.

[10] M. Corporation, "Telemetry data: A resource for automotive safety insights," MITRE, Tech. Rep. PR-23-2484, 2023, https://www.mitre.org/sites/default/files/2023-08/PR-23-2484-Telemetry-Data-A-Resource-for-Automotive-Safety-Insights.pdf.

[11] A. Consortium, "Layered software architecture," AUTOSAR, Tech. Rep. R22-11, 2022, https://www.autosar.org/fileadmin/standards/R22-11/CP/AUTOSAR_EXP_LayeredSoftwareArchitecture.pdf.

[12] A. Consortium, "Specification of diagnostic log and trace," AUTOSAR, Tech. Rep. R22-11, 2022, https://www.autosar.org/fileadmin/standards/R22-11/CP/AUTOSAR_SWS_DiagnosticLogAndTrace.pdf.

[13] M. Clemens, T. Müller, and R. Schäfer, "Event-driven fault diagnosis framework for automotive systems," US20110238258A1, U.S. Patent Application Publication, Sep. 29, 2011. [Online]. Available: https://patents.google.com/patent/US20110238258A1/en.

[14] M. Bertoncello, C. Martens, T. Möller, and T. Schneiderbauer, "Unlocking the full life-cycle value from connected-car data," McKinsey & Company, McKinsey Center for Future Mobility, Tech. Rep., Feb. 2021, White paper. [Online]. Available: https://www.mckinsey.com/industries/automotive-and-assembly/our-insights/unlocking-the-full-life-cycle-value-from-connected-car-data.

[15] M. Komorkiewicz, A. Chin, P. Skruch, and M. Szelest, "Intelligent data handling in current and next-generation automated vehicle development—a review," *IEEE Access*, vol. PP, 2023. DOI: 10.1109/ACCESS.2023.3258623.

[16] A. Theissler, J. Pérez-Velázquez, M. Kettelgerdes, and G. Elger, "Predictive maintenance enabled by machine learning: Use cases and challenges in the automotive industry," *Reliability Engineering & System Safety*, vol. 215, p. 107 864, 2021, ISSN: 0951-8320. DOI: https://doi.org/10.1016/j.ress.2021.107864. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0951832021003835.

[17] Ö. Özdemir, M. T. İşyapar, P. Karagöz, K. W. Schmidt, D. Demir, and N. A. Karagöz, *A survey of anomaly detection in in-vehicle networks*, 2024. arXiv: 2409.07505 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2409.07505.

[18] P. H. Brunheroto, A. L. Gonçalves Pepino, F. Deschamps, and E. de Freitas Rocha Loures, "Data analytics in fleet operations: A systematic literature review and workflow proposal," *Procedia CIRP*, vol. 107, pp. 1192–1197, 2022, Leading manufacturing systems transformation – Proceedings of the 55th CIRP Conference on Manufacturing Systems 2022, ISSN: 2212-8271. DOI: https://doi.org/10.1016/j.procir.2022.05.130. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2212827122004140.

[19] S. Tirupathi, D. Salwala, G. Zizzo, *et al.*, "Machine learning platform for extreme scale computing on compressed iot data," in *2022 IEEE International Conference on Big Data (Big Data)*, 2022, pp. 3179–3185. DOI: 10.1109/BigData55660.2022.10020540.

[20] J. Santos, M. Hempel, and H. Sharif, "Compression distortion-rate analysis of biomedical signals in machine learning tasks in biomedical wireless sensor network applications," in *2020 International Wireless Communications and Mobile Computing (IWCMC)*, 2020, pp. 746–751. DOI: 10.1109/IWCMC48107.2020.9148572.

[21] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948. DOI: https://doi.org/10.1002/j.1538-7305.1948.tb01338.x. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/j.1538-7305.1948.tb01338.x. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/j.1538-7305.1948.tb01338.x.

[22] K. Sayood, *Introduction to Data Compression*, 5th. Morgan Kaufmann Publishers Inc., 2017, ISBN: 978-0-12-809474-7.

[23] A. Thomasian, "Chapter 2 - storage technologies and their data," in *Storage Systems*, A. Thomasian, Ed., Morgan Kaufmann, 2022, pp. 89–196, ISBN: 978-0-323-90796-5. DOI: https://doi.org/10.1016/B978-0-32-390796-5.00011-5. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9780323907965000115.

[24] C. E. Shannon, "Communication in the presence of noise," *Proceedings of the I.R.E.*, vol. 37, no. 1, pp. 10–21, Jan. 1949. DOI: 10.1109/JRPROC.1949.232969.

[25] T. M. Cover and J. A. Thomas, *Elements of Information Theory, 2nd Edition*, 2nd. Hoboken, NJ: John Wiley & Sons, 2006, ISBN: 978-0-471-24195-9.

[26] A. Marchioni, A. Enttsel, M. Mangia, R. Rovatti, and G. Setti, "Anomaly detection based on compressed data: An information theoretic characterization," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 54, no. 1, pp. 23–38, 2024. DOI: 10.1109/TSMC.2023.10233222.

[27] C. E. Muniz-Cuza, S. K. Jensen, J. Brusokas, N. Ho, and T. B. Pedersen, "Evaluating the impact of error-bounded lossy compression on time series forecasting," in *Advances in Database Technology - EDBT*, ser. Advances in Database Technology - EDBT, OpenProceedings, Mar. 2024, pp. 650–663. DOI: 10.48786/edbt.2024.56.

[28] S. Ma, X. Zhang, C. Jia, Z. Zhao, S. Wang, and S. Wang, "Image and video compression with neural networks: A review," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 30, no. 6, pp. 1683–1698, Jun. 2020, ISSN: 1558-2205. DOI: 10.1109/tcsvt.2019.2910119. [Online]. Available: http://dx.doi.org/10.1109/TCSVT.2019.2910119.

[29] S. Johnsson, "Large scale efficient data readout for vehicle fleets," M.S. thesis, Chalmers University of Technology, 2025.

[30] N. Tishby, F. C. Pereira, and W. Bialek, "The information bottleneck method," *arXiv preprint physics/0004057*, 2000.

[31] N. Tishby and N. Zaslavsky, "Deep learning and the information bottleneck principle," in *2015 IEEE Information Theory Workshop (ITW)*, 2015, pp. 1–5. DOI: 10.1109/ITW.2015.7133169.

[32] R. Shwartz-Ziv and N. Tishby, "Opening the black box of deep neural networks via information," *arXiv preprint arXiv:1703.00810*, 2017.

[33] A. M. Saxe, Y. Bansal, J. Dapello, *et al.*, "On the information bottleneck theory of deep learning," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=ry_WPG-A-.

[34] A. A. Alemi, I. Fischer, J. V. Dillon, and K. Murphy, "Deep variational information bottleneck," *arXiv preprint arXiv:1612.00410*, 2016.

[35] D. J. Strouse and D. J. Schwab, "The deterministic information bottleneck," *Neural computation*, vol. 29, no. 6, pp. 1611–1630, 2017.

[36] G. Sun, P. Karras, and Q. Zhang, *Highly efficient direct analytics on semantic-aware time series data compression*, 2025. arXiv: 2503.13246 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2503.13246.

[37] N. Lai, D. A. Dewi, S. S. Maidin, W. Xiao, S. Zhao, and Q. Hu, "A comprehensive review of lightweight deep learning models for edge computing with future directions," *Discover Computing*, vol. 29, p. 110, 2026. DOI: 10.1007/s10791-026-10021-3.

[38] Z. Zheng and Z. Zhang, "A temporal convolutional recurrent autoencoder based framework for compressing time series data," *Applied Soft Computing*, vol. 147, p. 110 797, 2023, ISSN: 1568-4946. DOI: https://doi.org/10.1016/j.asoc.2023.110797. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1568494623008153.

[39] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, "Soundstream: An end-to-end neural audio codec," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 495–507, 2021.

[40] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," *arXiv preprint arXiv:2210.13438*, 2022.

[41] S. Ji, Z. Jiang, W. Wang, *et al.*, "Wavtokenizer: An efficient acoustic discrete codec tokenizer for audio language modeling," *arXiv preprint arXiv:2408.16532*, 2024.

[42] Y. Shang, G. Liu, R. R. Kompella, and Y. Yan, "Enhancing post-training quantization calibration through contrastive learning," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 15 921–15 930. DOI: 10.1109/CVPR52733.2024.01507.

A

Appendix 1